IUT Robert Schuman
Institut universitaire de technologie
Université de Strasbourg

P11 – Initiation à la programmation

-

Les procédures et les fonctions

Hervé L'helguen

Plan

1. Introduction

Introduction

Une procédure ou une fonction est un algorithme **annexe** qui assure de manière **autonome** un traitement particulier.

Quel est l'intérêt d'utiliser des procédures et des fonctions ?

- **Réutilisation du code** : éviter de réécrire plusieurs fois une même séquence d'actions dans l'algorithme. Le principe est d'écrire une fonctionnalité une seule fois et de la réutiliser plusieurs fois (DRY – Don't Repeat Yourself).
- **Lisibilité et maintenance** : si le programme principal contient tout le code il devient rapidement illisible. Il est plus facile de lire les appels (noms explicites) pour avoir une vue globale et d'aller voir les détails dans chaque procédure ou fonction (principe de la boite noire). La maintenance est ainsi facilitée, on va directement réparer le module en panne.

Introduction

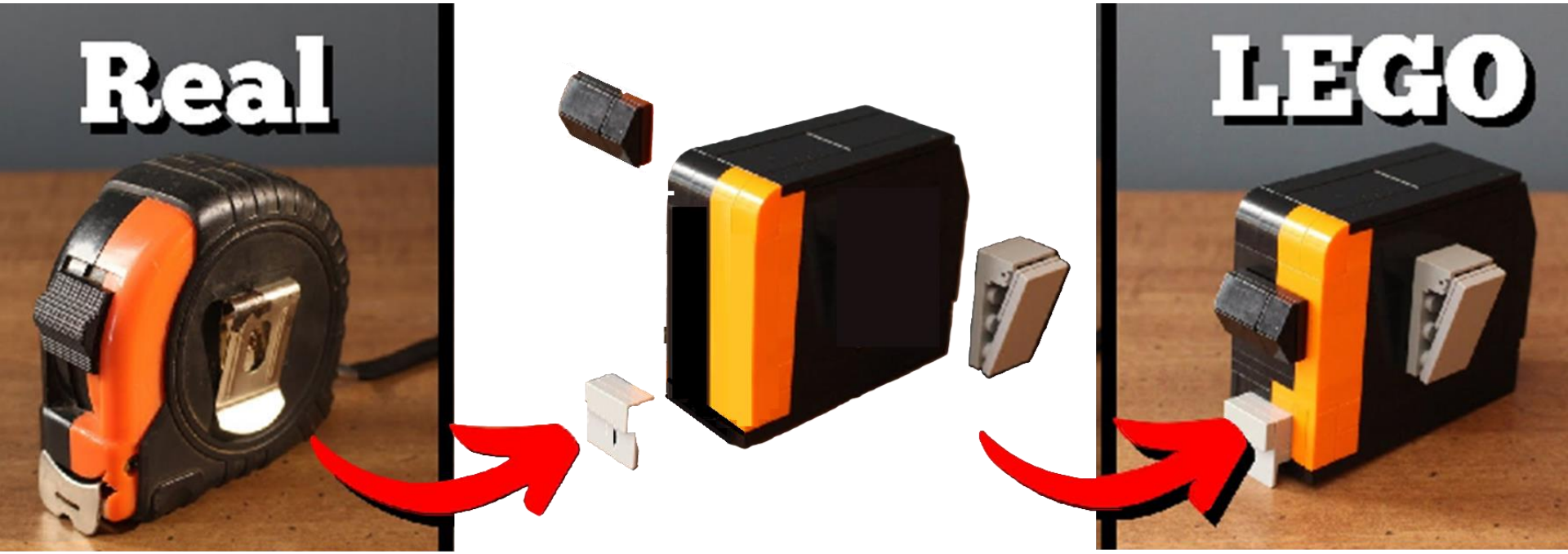
Quel est l'intérêt d'utiliser des procédures et des fonctions ?

- **Principe de modularité :** Consiste à construire et assembler une application en plusieurs parties indépendantes (des modules) :
  - Réduire la complexité (diviser un problème complexe en problèmes simples)
  - Isoler les responsabilités (chaque module est responsable de son traitement)
  - Faciliter les tests (si un test échoue, on peut facilement localiser le module qui pose problème)
  - Mettre facilement à jour (des prix, des taux, …)
  - Réutiliser un même module dans une autre application
  - Faciliter le travail à plusieurs
  - Faciliter l'évolution de l'application
  - Possibilité de développer une application (par exemple en Python) et d'appeler des fonctions écrites en C (interopérabilité). En SAE c'est ce que nous ferons…

Introduction

Quel est l'intérêt d'utiliser des procédures et des fonctions ?

- **Principe de modularité :**



Introduction

Quelle est la différence entre une procédure et une fonction ?

- Une procédure exécute une séquence d'actions **sans retourner de valeur**,
- Une fonction **détermine et retourne une valeur unique** à la partie de l'algorithme qui l'a appelée

Remarque : une procédure ou une fonction peut appeler d'autres procédures et fonctions. Une fonction peut même s'appeler elle-même…

Plan

1. Introduction
2. Procédures

Procédures

Je veux par exemple afficher la bannière suivante plusieurs fois dans un algorithme :

```
-----------------------------------------------
|   Top Programme V5.4 – Sept 2026    |
-----------------------------------------------
```

Plutôt que de réécrire le code qui permet cet affichage, je vais créer une procédure **`afficherBanniere()`**

Remarque : j'utilise camelCase pour nommer les procédures auxquelles je donne un nom explicite

Procédures

Algorithme de la procédure

```
Procédure afficherBanniere()
    Déclarations
Début
    Afficher "-----------------------------------"
    Afficher "|  Top Programme V5.4 – Sept 2026  |"
    Afficher "-----------------------------------"
Fin
```

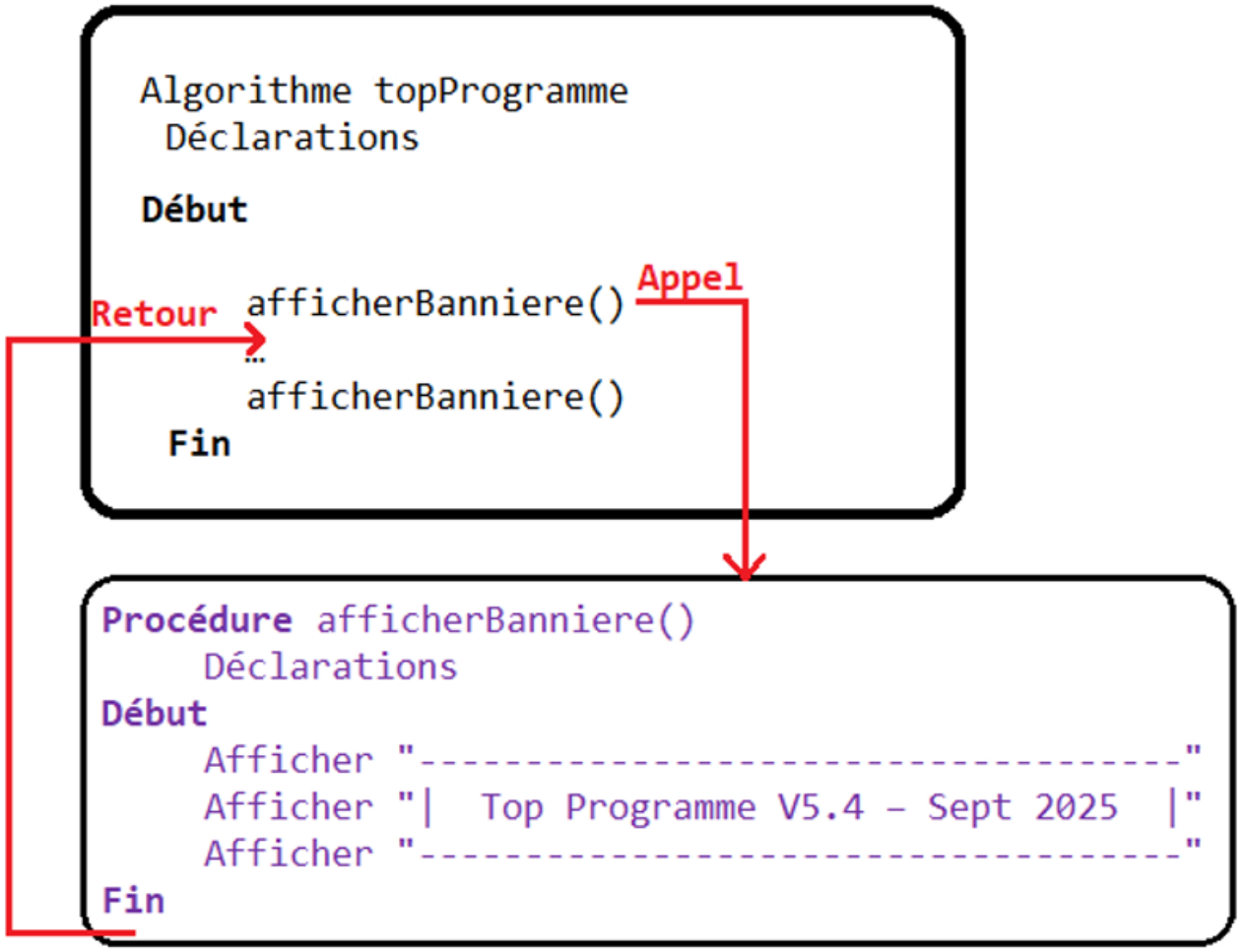
Procédures

L'appel dans l'algorithme principal :

```
Algorithme topProgramme
    Déclarations
Début
    //j'appelle la procédure simplement par son nom
    afficherBanniere()
    …
    afficherBanniere()
 Fin
```

Procédures

L'appel d'une procédure :



Plan

1. Introduction
2. Procédures
3. Paramètres et arguments

Paramètres et arguments

Lorsqu'une procédure a besoin d'informations pour s'exécuter, ces informations sont transmises sous forme d'arguments via des paramètres. Je veux par exemple créer une procédure qui affiche le montant en Euros d'une valeur en Francs. Pour fonctionner, la procédure a besoin de connaître la valeur en Francs :

```
Procédure afficherEuros(montantFr : réel)
    Déclarations
        Constantes
            VAL_EURO = 6.55956 : réel
        Variables
            montantEu : réel
Début
    montantEu = montantFr / VAL_EURO
    Afficher montantFr + " F = " + montantEu + " €"
Fin
```

Je déclare ici le paramètre qui va recevoir la valeur

Paramètres et arguments

Dans l'algorithme principal :

```
Algorithme conversionF_E
    Déclarations
        Variables
            montant : réel
 Début
    Afficher "Saisir le montant en francs : "
    Saisir montant
    //Appel de la procédure
    afficherEuro(montant)
Fin
```

Je fournis l'argument (la valeur) nécessaire à la procédure

Paramètres et arguments

L'appel d'une procédure paramétrée :

```
Algorithme conversionF_E
    Déclarations
        Variables
            montant : réel
Début
    Afficher "Saisir le montant en francs : "
    Saisir montant
    //Appel de l  procédure
    afficherEuros(montant)
Fin
```

Argument : contenu de montant

```
Procédure afficherEuros(montantFr: réel)
    Déclarations          Paramètre
        Constantes
            VAL_EURO = 6.55956 : réel
        Variables
            montantEu : réel
Début
    montantEu = montantFr / VAL_EURO
    Afficher montantFr + " F = " + montantEu + " €"
Fin
```

Constantes et variables locales

Paramètres et arguments

Il est bien entendu possible de passer plusieurs arguments à une procédure, par exemple avec cet appel :

```
afficherFiche(1, "Tarre", "Maggy", 2004)
```

Voici la déclaration de cette procédure :

```
Procédure afficherFiche(civilite : entier, nom : chaîne, prenom : chaîne, anneNais : entier)
```

Le nombre, l'ordre et le type des paramètres s'appelle la **signature**.
La signature de la procédure `afficherFiche()` est donc :

entier, chaîne, chaîne, entier

…et donc ce qui est attendu lors du passage des arguments.

Procédures en C#

```
class topProgramme                    //Programme principal
{
    static void Main()
    {
            //Déclarations
            string versionDate;

            versionDate = "V5.4 - Sept 2026";

            //Appel de la procédure
            AfficherBanniere(versionDate);
            …
            AfficherBanniere(versionDate);
     }

      //Codage de la Procédure
     static void AfficherBanniere(string version)
     {
       Console.WriteLine("--------------------------------");
       Console.WriteLine("| Top Programme" + version + " |";
       Console.WriteLine("--------------------------------"
     }
}
```

C'est le mot clé void "retour vide" qui indique qu'il s'agit d'une procédure

Remarque : Microsoft préconise de nommer les procédures et les fonctions en PascalCase en C#

Plan

1. Introduction
2. Procédures
3. Paramètres et arguments
4. Fonctions

Fonctions

Une fonction est une procédure spéciale chargée de déterminer et retourner **une valeur unique** à la partie de l'algorithme qui l'a appelée.

Dans une fonction **il n'y a pas d'affichage ni de saisie**, c'est l'algorithme principal qui s'en chargera au besoin !

- **Principe de responsabilité unique**, la fonction fait une seule chose (déterminer et retourner une valeur unique)
- **Principe de réutilisation**, sans affichage la fonction est réutilisable dans d'autres contextes
- **Principe de lisibilité et de maintenance**, séparer les calculs et l'affichage améliore la lisibilité de l'algorithme et facilite ses modifications ou mise à jour.

L'algorithme principal orchestre les appels aux fonctions et gère les interactions avec l'utilisateur.

Fonctions

Transformons la procédure afficherEuros() en fonction francsToEuros() qui va avoir la responsabilité de renvoyer le montant en Euros d'un montant reçu en Francs.

```
Fonction francsToEuros(montantFr : réel)
    Type de sortie : réel
    Déclarations
        Constantes
            VAL_EURO = 6.55956 : réel
        Variables
            montantEu : réel
Début
    montantEu = montantFr / VAL_EURO
    Retourner montantEu
 Fin
```

Je déclare le type de la valeur qui sera retournée par la fonction

Je retourne obligatoirement à l'algorithme principal la valeur déterminée par la fonction, cette valeur **doit être du type de sortie déclaré (ici réel)**

Fonctions

Dans l'algorithme principal :

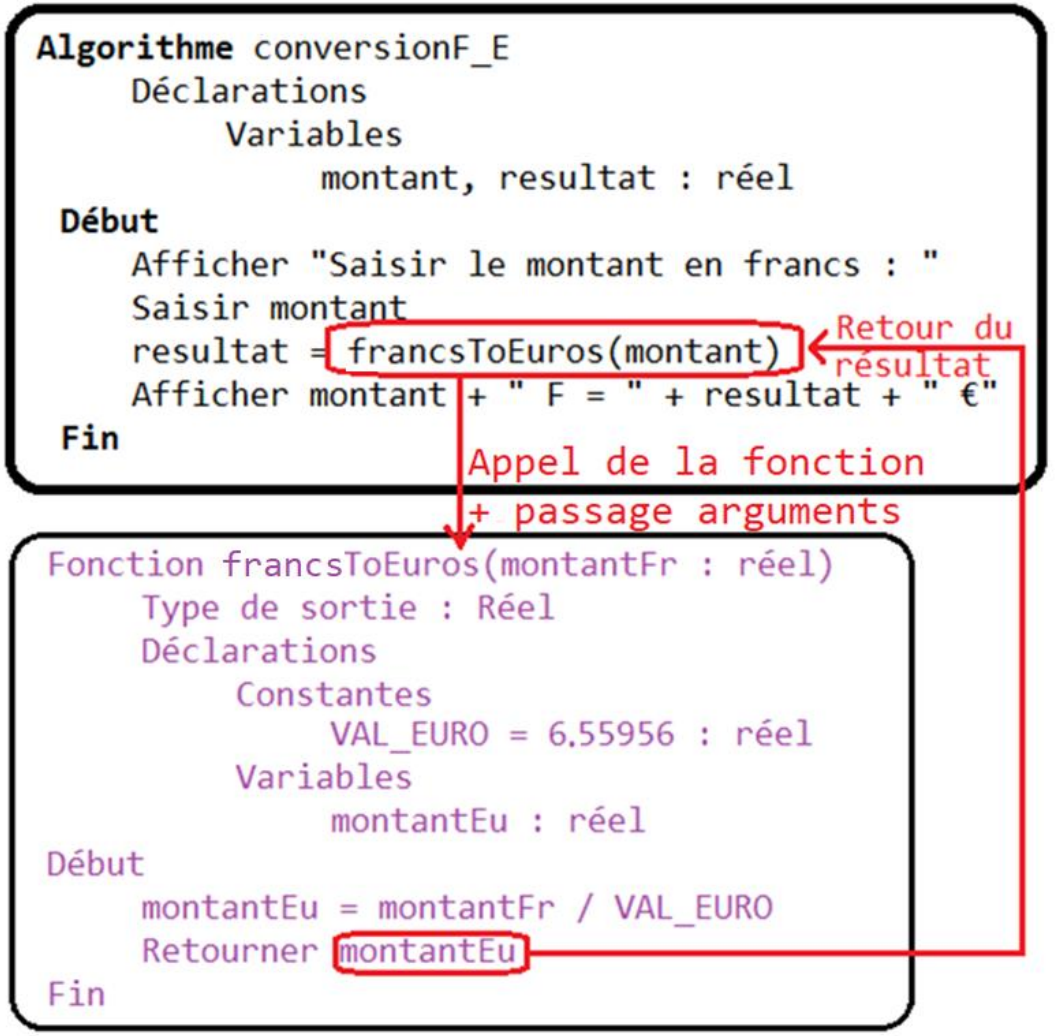
```
Algorithme conversionF_E
    Déclarations
        Variables
            montant, resultat : réels
 Début
    Afficher "Saisir le montant en francs : "
    Saisir montant
    //Appel de la fonction
    resultat = francsToEuros(montant)
    Afficher montant + " F = " + resultat + " €"
 Fin
```

Appel de la fonction

La saisie et l'affichage du résultat sont gérés dans l'algorithme principal

Fonctions

L'appel d'une fonction :



Fonctions

Important ! Du fait que la fonction est associée au retour d'une seule valeur, son appel se fait **toujours** dans un contexte :

- une affectation : `res = exempleFonction(10)`
- directement dans un calcul : `a = b * exempleFonction(c)`
- une comparaison : `Si e >= exempleFonction(a) Alors`
- lors d'un affichage :

```
Afficher "Résultat = " + exempleFonction(t)
```

- Un appel de fonction hors contexte n'a pas de sens :

~~`exempleFonction(z)`~~

Seule une procédure peut être appelée ainsi

Fonctions en C#

```
class conversion_E                    //Programme principal
{
    static void Main()
    {
        //Déclarations
        float montant, resultat;

        Console.Write("Saisir le montant en francs : ")
        montant = float.Parse(Console.ReadLine());

        //Appel de la fonction
        resultat = FrancsToEuros(montant);

        Console.WriteLine($"{montant} F = {resultat} €");
    }

    //Codage de la fonction
    static float FrancsToEuros(float montantFr)
    {
        const float VAL_EURO = 6.55956f;
        float montantEu;
        montantEu = montantFr * VAL_EURO;
        return montantEu; //retour de la valeur
    }
}
```

Le mot clé float (type autre que void) indique qu'il s'agit d'une fonction et que la valeur retournée par cette fonction sera obligatoirement de type float

Plan

1. Introduction
2. Procédures
3. Paramètres et arguments
4. Fonctions
5. Portée des variables et constantes

Portée des variables et constantes

Les déclarations effectuées dans l'algorithme principal ne sont pas connues des fonctions et procédures et vice-versa ! On dit qu'elles sont étanches.

Les constantes et les variables de la procédure ou de la fonction sont créées lors de son appel et détruites une fois que son exécution est terminée, leur durée de vie est temporaire.

Portée des variables et constantes

Les déclarations effectuées dans l'algorithme principal ne sont pas connues des fonctions et procédures et vice-versa ! On dit qu'elles sont étanches.

```
Algorithme  conversionF_E
    Déclarations
        Variables
            montant, resultat : réel
Début
    Afficher "Saisir le montant en francs : "
    Saisir montant
    resultat = FrancsToEuros(montant)
    Afficher montant + " F = " + resultat + " €"
Fin
```

```
Fonction francsToEuros(montantFr : réel)
    Type de sortie : Réel
    Déclarations
        Constantes
            VAL_EURO = 6.55956 : réel
        Variables
            montantEu : réel
Début
    montantEu = montantFr / VAL_EURO
    Retourner montantEu
Fin
```

Portée (ou scope)

Variables locales à conversionF_E

- montant
- resultat

Constante locale à francsToEuros

- VAL_EURO

Variables locales à francsToEuros

- montantFr (paramètre)
- montantEu

Plan

1. Introduction
2. Procédures
3. Paramètres et arguments
4. Fonctions
5. Portée des variables
6. Passage par référence

Passage par référence

Jusqu'à présent le passage de paramètre était réalisé "par valeur" :

```
Algorithme conversionF_E
    Déclarations
        Variables
            montant : réel
Début
    Afficher "Saisir le montant en francs : "
    Saisir montant
    //Appel de l  procédure
    afficherEuros(montant)
Fin
```

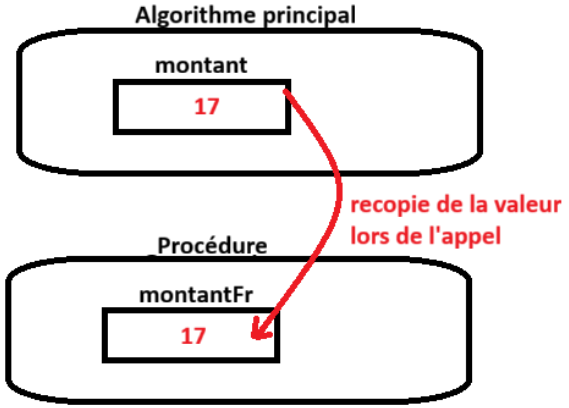
Argument : contenu de montant

```
Procédure afficherEuros(montantFr: réel)
    Déclarations          Paramètre
        Constantes
            VAL_EURO = 6.55956 : réel
        Variables
            montantEu : réel
Début
    montantEu = montantFr / VAL_EURO
    Afficher montantFr + " F = " + montantEu + " €"
Fin
```

Constantes et variables locales

Le contenu de la variable montant est recopié dans la variable montantFr le paramètre de la procédure.

montantFr est une variable locale de la procédure et complètement déconnectée de la variable montant de l'algorithme principal.



Passage par référence

Il peut y avoir intérêt à passer des arguments par référence, notamment si la fonction ou la procédure est chargée de modifier plusieurs valeurs.

Exemple : dans un algorithme principal je voudrais qu'une procédure frToEu() mette à jour directement la variable resultat :

```
Algorithme conversionF_E
    Déclarations
        Variables
            montant, resultat : réels
 Début
    Afficher "Saisir le montant en francs : "
    Saisir montant
    resultat = -1 //Valeur d'initialisation
    //Appel de la procédure
    frToEu(montant, référence resultat)
    Afficher montant + " F = " + resultat + " €"
 Fin
```

Passage par référence

Ecriture de la procédure frToEu() :

```
Procédure frToEu(montantFr : réel, référence res : réel)
    Déclarations
        Constantes
            VAL_EURO = 6.55956 : réel
Début
    res = montantFr / VAL_EURO
Fin
```

On verra plus tard que certains objets complexes sont systématiquement passés par référence.

Passage par référence

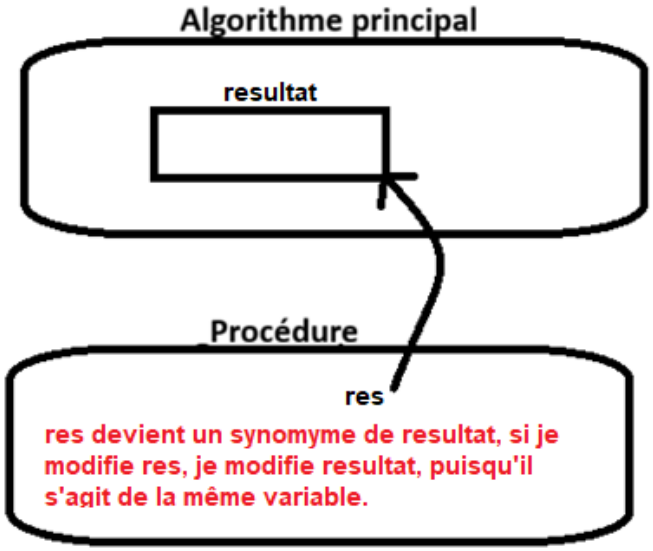
Si je décide de passer les arguments "par référence", le comportement global va être modifié :

```
Algorithme conversionF_E
      Déclarations
            Variables
                  montant, resultat : réels
Début
      Afficher "Saisir le montant en francs : "
      Saisir montant
      resultat = -1
      //Appel de la procédure
      francsToEuros(montant, référence resultat)
      Afficher montant + " F = " + resultat + " €"
Fin
```

recopie par valeur du montant en franc

```
Procédure frToEu(montantFr : réel, référence res : réel)
   Déclarations
      Constantes
         VAL_EURO = 6.55956 : réel
Début
      res = montantFr / VAL_EURO
Fin
```

Calcul du montant en euros et affectation dans la variable res qui est un synomyme (ou alias) de la variable resultat dans l'algorithme principal



res et resultat pointent vers la même zone mémoire tant que la procédure est en exécution.

Passage par référence

La portée des variables et des constantes est donc modifiée

```
Algorithme conversionF_E
      Déclarations
            Variables
                  montant, resultat : réels
 Début
      Afficher "Saisir le montant en francs : "
      Saisir montant
      resultat = -1
      //Appel de la procédure
      francsToEuros(montant, référence resultat)
      Afficher montant + " F = " + resultat + " €"
Fin
```

recopie par valeur du montant en franc

```
Procédure frToEu(montantFr : réel, référence res : réel)
   Déclarations
      Constantes
         VAL_EURO = 6.55956 : réel
Début
      res = montantFr / VAL_EURO
Fin
```

Calcul du montant en euros et affectation dans la variable res qui est un synomyme (ou alias) de la variable resultat dans l'algorithme principal

Portée (ou scope)

Variables locales à conversionF_E

montant

resultat

Constante locale à frToEu

VAL_EURO

Variables locales à francsToEuros

montantFr (paramètre)

res (synomyme ou alias de resultat durant l'exécution)

Passage par référence en C#

